

# Assesment for Learning

Advance Programming

Elvina Angelina  
Informatika (Kampus Kota Makassar)  
Universitas Ciputra Makassar  
eangelina04@student.ciputra.ac.id

## I. BRACKET CHECKER

### A. Problem Summary

Soal meminta sebuah program yang berfungsi untuk mengecek urutan tanda kurung dari `()`, `{}`, dan `[]` sudah balance atau tidak. Syarat seimbang ada dua: setiap kurung buka harus ditutup dengan jenis kurung tutup yang sama persis, dan urutannya tidak boleh saling menyilang. String yang kosong juga dihitung sebagai seimbang.

### B. Stack Design

**What gets pushed/popped and why:** Program ini hanya melakukan push (memasukkan data) untuk karakter kurung buka (`(`, `{`, atau `[`). Jika program membaca karakter kurung tutup, karakter itu tidak dimasukkan ke *Stack*. Sebaliknya, program langsung melakukan pop untuk mengambil kurung buka terakhir dari *Stack* agar bisa dicocokkan jenisnya dengan kurung tutup tersebut.

**How the stack represents the current state:** Stack di sini bertugas sebagai tempat ingatan sementara. Elemen yang paling atas (*top*) selalu mewakili kurung buka paling terakhir yang paling butuh dicarikan pasangan penutupnya segera.

### C. Algorithm Explanation

**Step-by-step logic:** Pertama, kita menggunakan perulangan (*for loop*) untuk mengecek teks karakter per karakter dari kiri ke kanan. Kalau ketemu kurung buka, masukkan ke *Stack*. Kalau ketemu kurung tutup, ambil (*pop*) elemen paling atas dari *Stack* dan cek apakah pasangannya sesuai. Kalau jenisnya berbeda (misalnya `"[)"`) dipasangkan dengan `"(")`, program langsung **return false**.

#### Handling of edge cases:

- Kasus kelebihan kurung tutup: Kalau ada kurung tutup tapi *Stack*-nya ternyata sudah kosong, program langsung mengembalikan nilai **false** agar tidak terjadi error.
- Kasus kelebihan kurung buka: Di akhir perulangan, program mengecek apakah *Stack* sudah kosong. Kalau ternyata masih ada isinya (`!stack.isEmpty()`), berarti ada kurung buka yang tidak ditutup, dan program akan mengembalikan nilai **false**.

### D. Correctness Argument

**Why the algorithm produces the required output:** Menggunakan algoritma *stack* pasti benar karena aturan urutan tanda kurung itu sama persis dengan sistem tumpukan barang (LIFO - Last-In, First-Out). Logikanya sederhana: tanda kurung yang terakhir kali dibuka, harus ditutup paling pertama.

**Explanation of the "twist requirement" and how your code satisfies it:** Syarat jebakan (*twist*) di soal ini adalah urutan kurung tidak boleh menyilang (misalnya `"{([])}"` itu salah), serta bagaimana jika pengguna memasukkan teks kosong atau hanya kurung buka saja.

- Mencegah persilangan: Tiap ketemu kurung tutup, program akan mengambil (*pop*) pucuk *Stack*. Kalau jenis pasangannya tidak cocok (misal tutupnya `]` tapi pucuk *Stack* isinya `(`), otomatis **me-return false**.
- Mencegah kurung menggantung: Di baris paling akhir, kode memanggil `return stack.isEmpty()`.

### E. Time & Space Complexity

**Time Complexity:  $O(n)$ .** Waktu prosesnya  $O(n)$  karena program cuma perlu membaca teks satu kali putaran dari kiri ke kanan (sebanyak  $N$  karakter). Perintah `push` dan `pop` di *Stack* juga dieksekusi secara instan ( $O(1)$ ).

**Space Complexity:  $O(n)$ .** Penggunaan memori bergantung pada panjang teks. Pada skenario terburuk, misalnya pengguna iseng mengetik kurung buka semua maka semua karakter tersebut terpaksa harus ditumpuk dan disimpan ke dalam memori *Stack*.

### F. Test Evidence

1. Enter a string of brackets: `{[()]}`  
The string is balanced.
2. Enter a string of brackets: `{[([])]}`  
The string is not balanced.
3. Enter a string of brackets: `(([[[]]]))`  
The string is balanced.
4. Enter a string of brackets: `[[]])`  
The string is not balanced.

### G. Code References

<https://github.com/eangelina04-dev/DS-Stack-Elvina-Angelina-2086022510009/blob/main/src/question1.java>

## II. REVERSE POLISH NOTATION (RPN) EVALUATION

### A. Problem Summary

Tugasnya adalah membuat fungsi yang bisa menghitung hasil dari urutan angka dan operator matematika (`+`, `-`, `*`, `/`). Bedanya dengan matematika biasa, dalam RPN: operator selalu diletakkan setelah operand. (contoh: `2 1 +` sama dengan `2 + 1`). Selain itu, hasil pembagian harus dibulatkan mendekati nol.

*Handwritten signature:*  
14/10  
10/3-2026



## B. Stack Design

**What gets pushed/popped and why:** Hanya angka (operand) yang di-push ke dalam Stack. Saat program membaca karakter operator (+, -, dll), operator tersebut tidak dimasukkan ke Stack. Sebaliknya, program akan men-pop dua angka paling atas dari Stack, menghitungnya, lalu men-push kembali hasil perhitungannya ke dalam Stack.

**How the stack represents the current state:** Stack di sini berfungsi untuk menyimpan angka-angka yang sedang mengantre untuk dihitung. Pucuk Stack selalu berisi angka terbaru atau hasil perhitungan sementara yang siap diproses dengan operator berikutnya.

## C. Algorithm Explanation

**Step-by-step logic:** Pertama, program memecah input teks menjadi potongan-potongan token berdasarkan spasi. Lalu, program mengecek setiap token dari kiri ke kanan. Kalau tokennya berupa angka, langsung masukkan ke Stack. Kalau tokennya berupa operator, ambil dua angka teratas dari Stack (angka pertama yang keluar jadi b, yang kedua jadi a), lakukan perhitungan sesuai operatornya (misal  $a + b$ ), lalu tumpuk lagi hasilnya ke dalam Stack. Setelah semua token habis dibaca, angka terakhir di dalam Stack adalah outputnya.

**Handling of edge cases:** Twist utamanya ada di urutan operasi pengurangan (-) dan pembagian (/). Saat kita melakukan pop dua kali, angka yang keluar pertama (b) sebenarnya adalah angka yang letaknya di sebelah kanan, dan angka yang keluar kedua (a) adalah yang di sebelah kiri. Oleh karena itu, kode secara spesifik menuliskannya sebagai  $a - b$  dan  $a / b$ , bukan kebalikannya, agar hasilnya tidak minus atau salah hitung.

## D. Correctness Argument

**Why the algorithm produces the required output:** format RPN (Reverse Polish Notation) mirip dengan struktur data Stack (LIFO). Dalam aturan RPN, sebuah simbol operator selalu mengeksekusi dua angka yang paling terakhir dibaca. Seperti Stack selalu mengumpulkan angka terbaru di pucuk tumpukan, setiap kali program kita membaca sebuah operator, proses pop secara alami akan menarik dua angka yang paling tepat dan paling baru untuk dihitung.

**Explanation of the "twist requirement" and how your code satisfies it:** Jebakan utama di RPN adalah urutan angka pada pengurangan/pembagian dan aturan pembulatan ke nol.

- Saat pop dilakukan, angka pertama yang keluar (b) diletakkan di kanan dan angka kedua (a) di kiri. Kode menjalankan  $a - b$  dan  $a / b$  agar hasilnya tidak terbalik.
- Aturan dibulatkan ke 0 langsung terpenuhi karena kita menggunakan tipe data int di Java, yang secara alami membuang angka di belakang koma tanpa perlu rumus tambahan.

## E. Time and Space Complexity

**Time Complexity:  $O(n)$ .** Waktu prosesnya sangat cepat karena program cuma perlu berjalan satu kali (satu loop) membaca urutan token dari awal sampai akhir. Proses

manipulasi Stack (seperti push dan pop) memakan waktu konstan atau  $O(1)$ .

**Space Complexity:  $O(n)$ .** Penggunaan memori bergantung pada jumlah token. Pada skenario terburuk (misalnya pengguna memasukkan rentetan angka semua tanpa ada operator satupun), maka semua angka tersebut harus disimpan dan ditumpuk di dalam memori Stack.

## F. Test Evidence

1. Input Tokens: 2 1 + 3 \*  
The value : 9

2. Input Tokens: 13 5 /  
The value : 2

3. Input Tokens: 10 5 \* 2 /  
The value : 25

4. Input Tokens: 3 2 / 4 +  
The value : 5

## G. Code References

<https://github.com/cangelina04-dev/DS-Stack-Elvina-Angelina-2086022510009/blob/main/src/question2.java>

## III. CHRONO STACK ENGINE

### A. Problem Summary

Tugasnya untuk membuat mesin waktu berkonsep Stack. Kita hanya punya tiga tombol: 1 (taruh angka 1), d (fotokopi angka paling atas), dan + (gabungkan dua angka teratas). Masalah utamanya adalah tombol + ini berbahaya. Setiap kali dipakai, sisa angka di bawahnya akan terkena "distorsi waktu" dan menyusut 1 angka. Tugas kita adalah menemukan urutan tombol agar hasil akhir angkanya sesuai dengan yang diminta soal.

### B. Stack Design

**What gets pushed/popped and why:** Dalam logika mesinnya, hanya angka yang di-push dan di-pop. Setiap kali kita menekan tombol +, dua elemen teratas di-pop untuk digabungkan, sementara elemen sisanya dipindahkan sementara ke wadah lain, dikurangi 1, lalu di-push kembali ke Stack utama.

**How the stack represents the current state:** Stack di sini ibarat sebuah timeline. Puncak Stack adalah masa kini" tempat kita menyusun angka baru, sedangkan elemen-elemen di bagian bawah Stack adalah "masa lalu" yang rentan menyusut setiap kali terjadi penggabungan angka di pucuknya.

### C. Algorithm Explanation

**Step-by-step logic:** Kita membangun dari bawah ke atas, algoritma kita justru berjalan mundur dari elemen target teratas (paling kanan input) menuju ke dasar (kiri). Program menggunakan fungsi pembantu untuk membedah cara tercepat membuat suatu angka: kalau angkanya genap, bagi







tersebut, alami membuang angka di belakang koma tanpa perlu rumus tambahan.

#### E. Time and Space Complexity

**Time Complexity:**  $O(n^2)$ . Terdapat loop luar sebesar  $O(n)$  kali, dan di dalamnya terdapat proses sortir yang memakan waktu  $O(n)$ .

**Space Complexity:**  $O(n)$ . Kapasitas memori yang digunakan berbanding lurus dengan jumlah elemen input yang disimpan di dalam Stack dan list sementara.

#### F. Test Evidence

1. 

```
Input:
42 9 17 63 28 5 74
Output:
5 9 17 28 42 63 74
```
2. 

```
Input:
34 7 89 12 56 3 71 25 44 18 90 6 39 52 14
Output:
3 6 7 12 14 18 25 34 39 44 52 56 71 89 90
```
3. 

```
Input:
1 2 3 5 4
Output:
1 2 3 4 5
```
4. 

```
Input:
34 51 20 1 67 54
Output:
1 20 34 51 54 67
```

#### G. Code References

<https://github.com/cangelina04-dev/DS-Stack-Elvina-Angelina-2086022510009/blob/main/src/question4.java>

### V. BOUNCING BOMBS

#### A. Problem Summary

Tugasnya adalah menghitung nilai momentum bom pada setiap pantulan. Aturan fisika sederhana yang diterapkan di sini adalah setiap kali bom memantul, momentumnya berkurang menjadi setengah dari nilai sebelumnya (pembagian bilangan bulat). Hasil yang diminta harus ditampilkan secara berurutan mulai dari pantulan terkecil (1) hingga kecepatan awal.

#### B. Stack Design

**What gets pushed/popped and why:** Program melakukan push angka momentum mulai dari yang terbesar ( $n$ ), lalu  $n/2$ , dan seterusnya hingga mencapai angka 1. Untuk menampilkan angkanya terbalik (dari kecil ke besar), kita memanfaatkan fungsi pop yang otomatis akan mengeluarkan angka yang paling terakhir dimasukkan (yaitu angka 1) terlebih dahulu.

**How the stack represents the current state:** Stack di sini berfungsi sebagai pengingat jejak pantulan. Puncak Stack selalu berisi momentum jatuh bom yang paling awal terjadi.

#### C. Algorithm Explanation

**Step-by-step logic:** Program menerima input kecepatan awal ( $n$ ). Kemudian program menggunakan perulangan while dengan membagi nilai momentum dengan 2 selama  $n$  masih di atas 0. Setelah perhitungan selesai cetak output dengan pop isi Stack satu per satu dari puncak.

**Handling of edge cases (Syarat jebakan & solusinya):** Program menggunakan pengecekan `!momentumStack.isEmpty()` di dalam loop pencetakan untuk memastikan tidak ada spasi berlebih di akhir angka terakhir.

#### D. Correctness Argument

**Why the algorithm produces the required output:** Algoritma ini menghasilkan output yang benar karena memanfaatkan sifat LIFO (*Last-In, First-Out*) milik Stack. Kita menghitung mundur jatuhnya bom (dari besar ke kecil), Stack secara otomatis membalikkan urutan tersebut saat kita mengeluarkannya, sehingga pengguna melihat urutan pantulan yang maju (dari kecil ke besar).

**Explanation of the "twist requirement" and how your code satisfies it**

- Soal meminta perhitungan momentum yang konsisten. Dengan menggunakan tipe data int, pembagian akan menghasilkan hasil yang dibulatkan langsung ke 0.

#### E. Time and Space Complexity

**Time Complexity:**  $O(\log n)$ . Setiap perluasan nilai ( $n$ ) dipotong menjadi setengahnya sehingga program pasti tidak akan melakukan perulangan sampai  $n$ .

**Space Complexity:**  $O(\log n)$ . Hanya menyimpan sejumlah angka hasil pembagian di dalam Stack.

#### F. Test Evidence

5. 

```
Input:
50
Output:
1 3 6 12 25 50
```
6. 

```
Input:
75
Output:
1 2 4 9 18 37 75
```
7. 

```
Input:
100
Output:
1 3 6 12 25 50 100
```
8. 

```
Input:
1000
Output:
1 1 7 15 31 62 125 250 500 1000
```

#### G. Code References

<https://github.com/cangelina04-dev/DS-Stack-Elvina-Angelina-2086022510009/blob/main/src/question5.java>

[1] Gemini AI